Manual técnico del prototipo Aurora

Requisitos, estructura del repositorio, arquitectura, configuración de Firebase, compilación e instalación, activación, panel web, contenido, pruebas, mantenimiento y seguridad operativa

Documento de la Fase 3 · Objetivo específico 3

Leydy Yohana Macareo Fuentes

Programa de Ingeniería de Sistemas, Universidad Autónoma de Bucaramanga

Proyecto de Grado II

Directora: Verónica Chajín Ortiz

Noviembre de 2026

Control del documento

Campo	Valor
Proyecto	Aurora: prototipo de aplicación móvil de uso discreto para el acceso a información, la orientación y el registro seguro ante la violencia de género en mujeres de Santander
Documento	Manual técnico del prototipo (Fase 3)
Versión del documento	1.0
Fecha	Noviembre de 2026
Autora	Leydy Yohana Macareo Fuentes
Directora	Verónica Chajín Ortiz
Versión del prototipo	v0.3.4-trl4
Normas de referencia	ISO/IEC/IEEE 26514 para la documentación de software; OWASP MASVS 2.1.0 para los controles de seguridad que se describen
Historial	1.0: versión inicial. Reúne las actividades y los resultados de la fase y el manual técnico: requisitos, estructura del repositorio, arquitectura, configuración de Firebase, compilación e instalación, activación, panel web, contenido, pruebas, mantenimiento, seguridad operativa y problemas frecuentes.

Tabla de contenido

Control del documento....2
Actividades y resultados de la fase....6
1. Propósito y alcance del manual....7
2. Requisitos....8
2.1 Para la aplicación Android....8
2.2 Para el panel web y los scripts....9
2.3 Cuenta y proyecto de Firebase....9
3. Estructura del repositorio....10
4. Arquitectura en resumen....11
4.1 Capas y módulos....11
4.2 Flujo de datos....12
4.3 Capas de seguridad....12
5. Configurar un proyecto de Firebase desde cero....13
5.1 Crear el proyecto....13
5.2 Authentication con correo....13
5.3 Firestore, reglas e índices....14
5.4 Llave de servicio para los scripts....15
5.5 Rol de administradora del panel....15
5.6 Plantilla del correo de recuperación (P-19)....16
6. Compilar e instalar la app....16
6.1 Preparar el JDK....16
6.2 Poner google-services.json....17
6.3 Compilar e instalar debug....17
6.4 Compilar release....17
6.5 El guardián de camuflaje....18
7. Activación de la app....19
7.1 Cómo funciona....19
7.2 La limitación de P-111....24
8. Panel web....25
8.1 Instalación y variables de entorno....25
8.2 Ejecutar, probar y construir....26
8.3 Despliegue....26
8.4 Arquitectura del panel....26
9. Contenido....27
9.1 Modelo de datos....27
9.2 Siembra con tools/seed....28

9.3 Publicación acotada de tipos (publicar-tipos)....28
9.4 Edición desde el panel: borradores y publicación....29
10. Pruebas....29
10.1 Comandos....29
10.2 Dónde están el plan y los resultados....30
10.3 Lo que no se puede automatizar....31
11. Mantenimiento....31
11.1 Inventariar cuentas....31
11.2 Eliminar una cuenta de prueba y sus datos....31
11.3 Limpiar borradores obsoletos del panel....32
11.4 Respaldos....32
12. Seguridad operativa....33
12.1 Reglas....33
12.2 Nada en logs....33
12.3 Secretos fuera del repositorio....33
12.4 Registro de cuentas (P-109)....34
13. Problemas frecuentes y cómo resolverlos....35
13.1 Gradle falla con «What went wrong: 27»....35
13.2 connectedDebugAndroidTest borró los datos del dispositivo....35
13.3 Un cambio de interfaz «no aparece» en el dispositivo....35
13.4 La compilación se detiene en google-services....36
13.5 El release falla en verificarCamuflajeDex....36
13.6 publicar-tipos o limpiar-borradores se quedan colgados....36
13.7 El script aborta diciendo que la llave está dentro del repositorio....36
13.8 El panel autentica pero no entra....36
13.9 El contenido nuevo no aparece en la app....37
13.10 El emulador entra en ANR y la app no arranca....37
13.11 No se puede capturar una pantalla del espacio protegido....37
13.12 Al automatizar por adb, el selector de documentos no responde....37
13.13 Las instrumentadas fallan «al azar» en el emulador de API 26....38
14. Pendientes antes de un piloto....38
14.1 Bloqueantes....39
14.2 Importantes....39
14.3 Tareas de documentación de la autora....40
14.4 Riesgos residuales aceptados....40
Glosario....42
Referencias....44

Anexos digitales..... 45

Actividades y resultados de la fase

La Fase 3 es el ciclo de implementación del diseño de investigación-acción del proyecto y corresponde al tercer objetivo específico: implementar los módulos de acceso a información, identificación preliminar de situaciones de violencia y registro organizado de incidentes, integrados con servicios en la nube. Comprende tres actividades: el desarrollo del módulo de acceso a información, con los tipos de violencia y las rutas de atención en Santander en lenguaje accesible; el desarrollo del módulo de identificación preliminar, con el cuestionario orientador y sus resultados, sin sustituir la valoración profesional; y el desarrollo del módulo de registro organizado de incidentes con integración en la nube, con almacenamiento cifrado, autenticación y control de acceso conforme a la Ley 1581 de 2012 y la norma ISO/IEC 27001 (International Organization for Standardization & International Electrotechnical Commission, 2022). Sus productos son la aplicación móvil funcional con los tres módulos integrados, el código fuente documentado y el manual técnico. Con ellos el proyecto alcanza el nivel de madurez TRL 4: prototipo validado en un entorno de laboratorio.

La implementación se hizo en iteraciones cortas, cada una cerrada con una versión etiquetada del prototipo y revisada con la directora del proyecto. Los tres módulos se construyeron sobre una base común de acceso y seguridad: la fachada de clima con datos reales de Open-Meteo, la entrada por la barra de búsqueda, la activación, la creación de la cuenta, el código de acceso y la guía de primer uso. Sobre esa base están el módulo de información, con tipos de violencia, rutas, entidades y líneas de atención disponibles sin conexión; el módulo de identificación preliminar, con el cuestionario y sus resultados orientativos, que se guardan solo si la usuaria lo confirma; y el módulo de registro, con adjuntos, sincronización cifrada entre dispositivos, exportación a PDF protegido con contraseña y recuperación desde otro teléfono. En paralelo se construyó el panel web para administrar el contenido y el cuestionario, y el contenido de la guía se cargó con información verificada contra fuentes oficiales.

El resultado de la fase es la versión v0.3.4-trl4 del prototipo, con sus 46 requerimientos funcionales y 11 no funcionales implementados, el panel web de administración, 330 commits en la rama principal del repositorio público leydymf/aurora-app y la documentación técnica del código: el registro de decisiones de arquitectura, la especificación de requerimientos, el plan y los resultados de las pruebas y el informe de seguridad. Este manual técnico es el tercer producto de la fase: describe lo que una persona con perfil técnico necesita para instalar, desplegar y mantener el prototipo sin haber participado en su construcción. El manual se organizó según la norma ISO/IEC/IEEE 26514 (International Organization for Standardization et al., 2022): indica a quién va dirigido, separa la información de referencia de los procedimientos paso a paso, señala con avisos de precaución los pasos que borran datos o escriben en el entorno publicado, reúne los problemas frecuentes con su solución y cierra con un glosario. Las instrucciones siguen la documentación oficial de Firebase y de Android (Google, 2026a, 2026b) y la de SQLCipher (Zetetic LLC, 2025), y los controles de seguridad que describe corresponden al estándar OWASP MASVS (OWASP Foundation, 2024). La verificación del prototipo (pruebas técnicas, evaluación de seguridad y evaluación de usabilidad) se presenta en el documento de la Fase 4.

1. Propósito y alcance del manual

Este documento está dirigido a una persona con perfil técnico que reciba el proyecto y deba instalarlo, desplegarlo y mantenerlo sin haber participado en su construcción. Explica qué herramientas hacen falta, qué hay en cada carpeta, cómo levantar un proyecto de Firebase desde cero, cómo compilar e instalar la aplicación, cómo administrar el contenido, cómo correr las pruebas y qué queda pendiente antes de usar la aplicación con personas reales.

Lo que no cubre:

- El detalle de arquitectura y las decisiones de diseño. Están en `docs/ARQUITECTURA.md`, que incluye el registro de decisiones (ADR) y es la referencia obligatoria antes de tocar código.
- Los requisitos con sus criterios de aceptación (`docs/REQUERIMIENTOS.md`) ni los casos de prueba (`docs/PLAN_PRUEBAS.md`, `docs/RESULTADOS_PRUEBAS.md`).
- La evaluación de seguridad (`docs/INFORME_SEGURIDAD_OWASP.md`) ni el manual de usuaria.

Dos condiciones aplican a todo el manual:

1. El repositorio es público por decisión de la autora. Este manual no contiene credenciales, identificadores de proyecto, correos reales ni el código de activación: donde hace falta un valor propio aparece un marcador como `<ID_DEL_PROYECTO>`. El repositorio sí contiene algunos de esos datos, por decisión y declarados en `docs/INFORME_SEGURIDAD_OWASP.md` §10.11 y en P-108: el identificador del proyecto de Firebase (en `admin/README.md`, `tools/admin/README.md`, `firestore/README.md` y otros), el correo de contacto de la autora (`README.md`) y el código de activación del build de depuración (`app/src/debug/`, y citado en varios documentos). Lo que no está expuesto, verificado sobre el árbol y sobre el historial completo, es `google-services.json`, las llaves de firma, la llave de servicio y los `.env`. Nada de lo expuesto es reutilizable en una implementación real, precisamente por lo que dice el punto 2.
2. Aurora es un prototipo académico de nivel TRL 4. Su camuflaje (nombre, ícono, nombre de paquete, apariencia y mecanismo de entrada) es público y por lo tanto ya no oculta la aplicación: una implementación para uso real debe rehacerlo por completo en un repositorio nuevo y privado. Ver §14 y P-108 en `docs/PENDIENTES.md`.

2. Requisitos

2.1 Para la aplicación Android

Herramienta	Versión que usa el proyecto	Nota
JDK	21 (se usó el JBR 21.0.4 que trae Android Studio)	Gradle 8.10.2 no arranca con JDK 27; ver §13.1
Gradle	8.10.2	Incluido en el repositorio (`gradlew`); no hace falta instalarlo
Android Gradle Plugin	8.8.0	Fijado en `gradle/libs.versions.toml`
Android Studio	Ladybug Feature Drop (2024.2.2) o posterior	Es la línea compatible con AGP 8.8

Herramienta	Versión que usa el proyecto	Nota
Android SDK Platform	API 35 (`compileSdk` y `targetSdk` = 35)	`minSdk` = 26 (Android 8.0)
Android SDK Build-Tools	35.0.0 o posterior	
Kotlin	2.0.0	

Dependencias destacadas, todas resueltas por Gradle: Jetpack Compose (BOM 2025.02.00), Material 3, Hilt 2.51.1, Room 2.6.1, SQLCipher 4.9.0, Firebase BOM 33.16.0, Retrofit 2.11.0, PdfBox-Android 2.0.27.0. El catálogo completo, con el motivo de cada versión fijada, está en `gradle/libs.versions.toml`.

Para probar en dispositivo se necesita además `adb` y, si no hay teléfono, un emulador. En este proyecto `adb` vive en `~/Library/Android/sdk/platform-tools/adb` (macOS) y no suele estar en el `PATH`.

2.2 Para el panel web y los scripts

Herramienta	Versión	Nota
Node.js	20 o posterior (probado con 22.21.0)	Lo usan `admin/`, `tools/seed/` y `tools/admin/`
npm	10 o posterior (probado con 10.9.4)	
Firebase CLI (`firebase-tools`)	Opcional	Solo para desplegar reglas por línea de comandos y para el emulador de Firestore. Nunca se instaló en el entorno de desarrollo: las reglas se desplegaron siempre desde la consola web

2.3 Cuenta y proyecto de Firebase

Hace falta una cuenta de Google y un proyecto de Firebase propio con:

- Authentication con el proveedor *Correo electrónico/contraseña*.
- Cloud Firestore en modo nativo.
- Plan Spark (gratuito). El proyecto se diseñó para no salir de la capa gratuita: no usa Firebase Storage, Functions ni Hosting, y las evidencias adjuntas viajan troceadas por Firestore precisamente para evitar Storage, que exige plan de pago.

También hace falta una llave de servicio (cuenta de servicio) para los scripts de `tools/`, que debe vivir fuera del repositorio (§5.4).

3. Estructura del repositorio

```
aurora/
├── app/                        Aplicación Android (único módulo Gradle)
│   ├── build.gradle.kts        Configuración del módulo + guardián del camuflaje
del APK
│   ├── proguard-rules.pro      Reglas de R8 para el release
│   ├── google-services.json    Configuración de Firebase (NO versionado; §6.2)
│   └── src/
│       ├── main/               Código y recursos de producción
│       ├── debug/              Solo builds debug: siembra del código de
activación
│       │                       y retiro de la PreviewActivity exportada
│       ├── test/               Pruebas de JVM (JUnit, Robolectric) y los
guardianes
│       │   └── resources/      Lista de términos vedados y línea base del
guardián del APK
│       └── androidTest/        Pruebas instrumentadas (Keystore, SQLCipher,
export)
├── admin/                      Panel web de administración (Vite + React +
TypeScript)
│   ├── src/                    Código del panel
│   ├── .env.example            Plantilla de la configuración del SDK web
│   └── README.md               Guía propia del panel
├── tools/
│   ├── seed/                   Siembra del contenido en Firestore (Node +
firebase-admin)
│   │   ├── content/*.json      Contenido que se siembra
│   │   ├── source/*.md         Fuentes del contenido (origen de los JSON)
│   │   ├── seed.mjs            Siembra completa
│   │   └── publish-topics.mjs Publicación acotada de tipos concretos
│   └── admin/                  Scripts administrativos (Node + firebase-admin)
│       ├── create-admin.mjs        Alta de la administradora con el claim
`admin`
│       ├── listar-cuentas.mjs      Inventario de cuentas de Auth (solo
lectura)
│       ├── delete-test-account.mjs Eliminación de una cuenta y sus datos
│       └── limpiar-borradores.mjs  Retiro de borradores obsoletos del panel
├── firestore/
│   ├── README.md               Cómo desplegar y verificar las reglas (CP-37)
│   └── rules.test.mjs          Pruebas de las reglas (requieren el emulador)
├── firestore.rules             Reglas de seguridad de Firestore
├── docs/
│   ├── ARQUITECTURA.md         Reglas de arquitectura y registro de decisiones
(ADR)
│   ├── REQUERIMIENTOS.md       46 RF y 11 RNF con criterios de aceptación
│   ├── PLAN_PRUEBAS.md         Casos de prueba CP-01..CP-39
│   ├── RESULTADOS_PRUEBAS.md Resultados automatizados y corridas manuales
│   ├── PENDIENTES.md           Tareas abiertas y riesgos residuales (P-xx)
│   ├── DECISIONES_DEL_PROYECTO.md Decisiones del proyecto
│   ├── INFORME_SEGURIDAD_OWASP.md Evaluación MASVS/MASTG
│   ├── RESUMEN_SEGURIDAD_TESIS.md Resumen de seguridad para el informe
│   ├── MANUAL_TECNICO.md       Este documento
│   ├── PREPARACION_SESIONES.md Preparación de los teléfonos de evaluación
│   ├── mockups/, mockups-v2/ Diseños (v2 es el vigente)
│   ├── marca/, manual-de-marca.pdf Identidad visual
│   └── evidencias-owasp/       Evidencias de la evaluación de seguridad
├── .claude/agents/             Definición de los tres agentes de revisión
├── CLAUDE.md                   Contexto y reglas de trabajo del proyecto
├── PLAN_FASE3.md, PLAN_POST_MVP.md, PLAN_UI_V2.md  Planes históricos
```

```
└── README.md                  Presentación del repositorio
```

Solo `app/` es un módulo Gradle (`settings.gradle.kts` incluye `:app` y nada más). El panel web no forma parte del APK y Gradle no lo compila.

4. Arquitectura en resumen

El detalle, con el motivo de cada decisión, está en `docs/ARQUITECTURA.md`. Aquí va solo lo necesario para orientarse.

4.1 Capas y módulos

MVVM con Clean Architecture en tres capas, organizado por feature:

```
app/src/main/java/com/auroraclima/app/
├── core/            Transversal: security, database, network, catalog,
directory, ui, startup
├── weather/         Fachada de clima (RF-01..RF-04)
├── access/          Entrada por código, Cómo usar Aurora, Ajustes (RF-05..RF-
16)
├── accounts/        Cuenta, consentimiento, sincronización de llaves (RF-07,
RF-17..RF-20, RF-46)
├── content/         Guía de orientación (RF-21..RF-26)
├── questionnaire/   Cuestionario y su resultado (RF-27..RF-31)
└── records/         Registro de hechos, adjuntos y exportación (RF-32..RF-39,
RF-45)
```

Cada feature se divide en `ui/` (pantallas Compose + ViewModel), `domain/` (modelos, contratos de repositorio y casos de uso; Kotlin puro) y `data/` (implementaciones, DAO, DTO, fuentes remotas).

La regla de dependencias es obligatoria: `ui` → `domain` ← `data`. La UI nunca importa de `data`; `domain` no conoce Android, Room ni Firebase. Las features no se importan entre sí: lo común vive en `core/` o se comunica por navegación. Cuando dos features deben cooperar, se hace por un contrato en `core` — `SearchQueryGate` (la barra de búsqueda consultando la puerta de acceso), `DirectoryProvider` y `CategoryCatalog` (el cuestionario y el registro leyendo el contenido publicado).

Inyección de dependencias con Hilt en todo el árbol. El estado de cada pantalla es un `data class` inmutable expuesto como `StateFlow`. Las operaciones de `data` devuelven

`Result<T>`: la UI nunca recibe una excepción y ningún mensaje de error visible puede revelar el espacio protegido.

4.2 Flujo de datos

- **Fachada:** Retrofit consulta la API de Open-Meteo (sin llave de API) y la caché de clima vive en una base Room sin cifrar y propia del feature —una base de clima en claro es lo esperable en una app meteorológica.
- **Contenido de la guía y cuestionario:** se leen de Firestore con persistencia offline activada. Esa caché es la única copia local del contenido; no se espeja en la base cifrada.
- **Datos de la usuaria:** se escriben en una base Room cifrada con SQLCipher (`journal_store.db` para el registro, `guidance_store.db` para los resultados guardados) y se respaldan en Firestore ya cifrados en el cliente: cada documento lleva solo `payload`, `iv` y `schemaVersion`. El proyecto de Firebase nunca ve texto en claro.
- **Adjuntos:** se guardan en `filesDir/media_store/` con nombre opaco y contenido sellado por stream; el mapeo registro ↔ archivo vive cifrado en la base. Para el respaldo se re-cifran y se trocean en documentos de Firestore. `KeystoreMediaCipher` nunca entrega al Keystore más de 32 KB por llamada (en Android 8.0 y 8.1 una llamada mayor de unos 128 KB se truncaba sin error), y `AttachmentStore` abre cada adjunto completo después de sellarlo y compara su SHA-256 con lo escrito: si no coincide, borra el archivo y la importación falla con un aviso propio. Detalle en `docs/INFORME_SEGURIDAD_OWASP.md` §11.

4.3 Capas de seguridad

Mecanismo	Dónde vive
`FLAG_SECURE` en toda pantalla protegida, aplicado por construcción al registrar la ruta en `secureGraph`	`core/security/SecureNavigation.kt`
Envoltorios para las ventanas propias, que no heredan la bandera: `SecureDialog`, `SecureSheet`, `SecureDayPicker`	`core/security`
`SecureTextEntry`: quita «copiar/cortar» del menú de selección y pide al teclado no aprender ni sugerir	`core/security`
Protección de la miniatura de recientes mientras se teclea en la barra (API 33+)	`core/security/RecentsThumbnailGuard`
Sesión única, cierre por inactividad (5 min), por segundo plano y salida rápida; limpieza por multibinding de `SensitiveStateCleaner`	`core/security/SessionManager`

Mecanismo	Dónde vive
Comparación del hash del código en tiempo constante (`constantTimeEquals`); por debajo de la longitud mínima ni se hashea, de modo que la igualdad de tiempos vale entre candidatos de longitud comparable —que es el caso que RF-13 tiene que igualar: nombre de ciudad contra clave—. Cualquier fallo interno degrada a «búsqueda sin resultados»	`access/domain/usecase/ValidateSecretCodeUseCase`
Seis llaves AES-GCM no exportables del Android Keystore, una por responsabilidad: `local_index_key` (hash del código), `local_store_key` (passphrase de SQLCipher), `local_media_key` (adjuntos), `local_backup_key` (DEK local), `local_account_key` (credenciales del reingreso), `local_sync_key` (legado, se conserva solo para poder destruirlo)	`core/security`, `core/database` (`DatabaseKeyManager`), `access/data` (`KeystoreSecretCodeHasher`), `accounts/data` (`LocalCredentialStore`)
Respaldo recuperable: DEK aleatoria envuelta por una llave derivada de la contraseña con PBKDF2-HMAC-SHA256, 600 000 iteraciones	`core/security/PasswordKeyWrapper` + `PasswordKeyDerivation`, orquestado por `SyncKeyManager`
Separación de accesos en la nube: `firestore.rules`	raíz del repositorio
Guardianes automáticos de camuflaje y de reglas de arquitectura	`app/src/test` y la tarea `verificarCamuflajeDex`

Regla permanente: nada del espacio protegido se registra en logs, crashlytics ni analítica. El proyecto no incluye ninguna biblioteca de analítica de terceros, y en release las llamadas a `android.util.Log` se eliminan.

5. Configurar un proyecto de Firebase desde cero

5.1 Crear el proyecto

1. console.firebase.google.com → *Agregar proyecto*. El identificador que asigne la consola es el que en este manual se escribe `<ID_DEL_PROYECTO>`.
2. Desactivar Google Analytics. El proyecto no debe tener analítica (regla 4 de `CLAUDE.md`).
3. Dejar el proyecto en plan Spark. Ver §12.4.

5.2 Authentication con correo

1. *Compilación* → *Authentication* → *Comenzar*.
2. *Sign-in method* → habilitar Correo electrónico/contraseña. Dejar desactivado el vínculo por correo (passwordless).

3. *Settings → Public-facing name*: ponerlo con identidad de clima (por ejemplo «Aurora Clima»). Es el valor que Firebase inserta en `%APP_NAME%` en los correos, y no se controla desde la plantilla.
4. La aplicación no exige verificar el correo: `CreateAccountUseCase` valida el formato y la longitud mínima de contraseña (8 caracteres) en el dispositivo y llama a `createUserWithEmailAndPassword`. No se envía ningún correo de verificación.

5.3 Firestore, reglas e índices

Precaución. Mientras no se desplieguen las reglas del paso 2, cualquier persona puede leer y escribir la base de datos. Las reglas se despliegan antes de crear cuentas o de sembrar contenido.

1. *Compilación → Firestore Database → Crear base de datos →* modo producción y la región más cercana.
2. **Desplegar las reglas.** Son el archivo `firestore.rules` de la raíz del repositorio. Mientras no se desplieguen, la base queda abierta. - Desde la consola: *Firestore → Reglas →* pegar el contenido completo de `firestore.rules` → revisar el diff → *Publicar*. - Con Firebase CLI: `firebase deploy --only firestore:rules` desde la raíz. Requiere crear un `firebase.json` que apunte a `firestore.rules`; ese archivo no existe en el repositorio porque el despliegue siempre se hizo por consola.
3. **Verificar las reglas.** Los ocho casos de la tabla del *Rules playground* están en `firestore/README.md`; el que demuestra RNF-06 es el de una cuenta con el claim `admin` intentando leer `users/<uid>/entries/<id>`, que debe quedar denegado. Con el emulador instalado: ``bash firebase emulators:exec --only firestore "node --test firestore/rules.test.mjs" ``
4. **Índices.** No hay que crear ninguno. La aplicación y el panel no usan consultas compuestas —leen colecciones completas o documentos por ruta y ordenan en memoria por el campo `order`—, así que el repositorio no declara `firestore.indexes.json`. Si en el futuro se agrega una consulta con `where` y `orderBy`, Firestore devolverá un error con un enlace para crear el índice.

Lo que protegen las reglas:

- `topics`, `routes`, `agencies`, `lines`, `forms`: lectura para cualquier cuenta autenticada; escritura solo con el custom claim `admin == true`.
- `drafts` (borradores del panel): lectura y escritura solo con el claim `admin`. La aplicación nunca consulta esta colección.
- `users/{uid}/`: lectura y escritura exclusivas de su dueña. El claim `admin` no otorga ningún acceso aquí.
- Todo lo demás, denegado.

5.4 Llave de servicio para los scripts

Los scripts de `tools/` usan `firebase-admin`, que omite las reglas por diseño.

1. Consola → ⚙️ *Configuración del proyecto* → *Cuentas de servicio* → Generar nueva clave privada.
2. Guardarla fuera del repositorio, en la ruta canónica del proyecto: ``bash mkdir -p ~/.config/aurora-seed mv ~/Downloads/<archivo-descargado>.json ~/.config/aurora-seed/service-account.json chmod 600 ~/.config/aurora-seed/service-account.json ``
3. Esa llave da acceso administrativo total al proyecto. Nunca se copia dentro del repositorio ni se envía por correo o chat. Hay tres barreras: la ruta canónica está fuera del árbol, `.gitignore` veda `service-account*.json` y `~-adminsdk-~.json`, y cada script aborta si la ruta de la llave cae dentro del repositorio.
4. Rotarla o revocarla desde *IAM* → *Cuentas de servicio* si se sospecha exposición, y al terminar el piloto.

Todos los scripts la leen de la variable de entorno:

```
export GOOGLE_APPLICATION_CREDENTIALS="$HOME/.config/aurora-seed/service-account.json"
```

5.5 Rol de administradora del panel

No hay registro público. La cuenta de administradora se crea con un script, que además le asigna el custom claim `admin=true` —la única forma de dar ese rol:

```
cd tools/admin
npm ci
export GOOGLE_APPLICATION_CREDENTIALS="$HOME/.config/aurora-seed/service-account.json"

# Contraseña temporal generada por el script (se imprime una sola vez):
node create-admin.mjs --email <correo-admin>

# O fijándola (mínimo 8 caracteres):
node create-admin.mjs --email <correo-admin> --password '<contraseña-fuerte>'
```

- Es idempotente: reejecutarlo solo reafirma el claim. Sin `--password` no toca la contraseña de una cuenta existente; con `--password` la restablece.
- Tras asignar el claim hay que volver a iniciar sesión para que el token lo lleve. El panel fuerza el refresco del token al entrar, así que basta con eso.
- Verificar el claim: ``bash node -e "import('firebase-admin/app').then(async m=>{m.initializeApp({credential:m.applicationDefault()});const {getAuth}=await import('firebase-admin/auth');const u=await getAuth().getUserByEmail('<correo-

```
admin>');console.log(u.customClaims)})"   #   Debe   imprimir:
{ admin: true } ``
```

5.6 Plantilla del correo de recuperación (P-19)

La aplicación ofrece recuperación de contraseña por correo (RF-46). El mensaje lo compone Firebase y, con la plantilla por omisión, nombra el proyecto y usa un remitente del dominio de Firebase: un rastro fuera del dispositivo, en una bandeja de entrada que cualquiera puede abrir. Eso compromete el camuflaje.

No se debe entregar un APK con esa pantalla a ninguna usuaria antes de personalizar la plantilla. Pasos:

1. Consola → *Authentication* → *Templates* → *Password reset*.
2. Reemplazar remitente, asunto y cuerpo por el texto con identidad de clima aprobado en `docs/DECISIONES_DEL_PROYECTO.md` §4.2.
3. Confirmar que *Authentication* → *Settings* → *Public-facing name* dice lo mismo (§5.2, punto 3): es lo que Firebase mete en `%APP_NAME%`, y la plantilla no lo cubre.
4. Probar con un correo real y guardar captura del mensaje recibido y de la página de restablecimiento que abre el enlace —esa página la aloja Firebase y muestra su propio título y dominio. Dejar constancia de qué remitente y qué dominio ve la usuaria.

Detalle en P-19 y P-88 de `docs/PENDIENTES.md`.

6. Compilar e instalar la app

6.1 Preparar el JDK

Gradle 8.10.2 falla con el JDK 27 (§13.1). Lo más simple es apuntar `JAVA_HOME` al JDK que trae Android Studio:

```
export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"
# macOS
java -version   # debe decir 21.x
```

Android Studio usa ese JDK por su cuenta; la variable solo hace falta para invocar `./gradlew` desde la terminal.

6.2 Poner google-services.json

El archivo no está versionado (`.gitignore` lo veda) porque es propio de cada proyecto de Firebase. Para obtener el propio:

1. Consola → ⚙️ *Configuración del proyecto* → *Tus apps* → Agregar app → Android.
2. Nombre del paquete: `com.auroraclima.app` (el `applicationId` del proyecto; si se cambia, hay que cambiarlo también en `app/build.gradle.kts`).
3. Descargar `google-services.json` y colocarlo en `app/google-services.json`.

Sin ese archivo el plugin `com.google.gms.google-services` detiene la compilación.

6.3 Compilar e instalar debug

```
./gradlew assembleDebug            # APK en app/build/outputs/apk/debug/app-
debug.apk
./gradlew installDebug             # compila e instala en el dispositivo
conectado
```

`assembleDebug` no instala nada. Si se cambió algo de la interfaz y hay que mirarlo en el dispositivo, instalar y comprobar la hora de instalación antes de sacar conclusiones:

```
adb install -r app/build/outputs/apk/debug/app-debug.apk
adb shell dumpsys package com.auroraclima.app | grep lastUpdateTime
```

6.4 Compilar release

```
./gradlew assembleRelease          # APK en app/build/outputs/apk/release/app-
release.apk
```

El release lleva R8 con ofuscación y reducción de recursos activadas (`isMinifyEnabled` e `isShrinkResources`), con las reglas de `app/proguard-rules.pro`. Es obligatorio: ningún artefacto debe salir del entorno con los nombres del espacio protegido legibles en el DEX.

Firma (P-91). Hoy el release se firma con el certificado de depuración (`CN=Android Debug`, solo esquema v2), decisión de TRL 4 documentada en `app/build.gradle.kts`. El almacén de depuración usa una contraseña universalmente conocida y vive sin protección en

`~/.android/`: quien lo obtenga puede firmar una actualización que el dispositivo acepta como la misma aplicación. Antes de entregar un APK a cualquier persona hay que generar un keystore propio fuera del repositorio (`.gitignore` ya veda `~.jks` y `~.keystore`), declarar un `signingConfig` de release y firmar con los esquemas v2 y v3.

Versión. `versionCode = 1` y `versionName = "1.0"` nunca se subieron; la versión real del proyecto la llevan las etiquetas de git (`v0.3.4-trl4` es la vigente). Si el proyecto va a distribuirse, hay que empezar a versionar el `build.gradle.kts`.

6.5 El guardián de camuflaje

Hay dos niveles de verificación automática del camuflaje:

1. Sobre las fuentes y los recursos, en las pruebas de JVM: `-CamouflageResourcesTest` barre `strings.xml`, los literales de Kotlin, los nombres de constantes de enum y las plantillas de `toString()` que R8 conserva, contra la lista de `app/src/test/resources/camuflaje-terminos-vedados.txt` (fuente única). Lleva su propia lista de identificadores aceptados, igual que el guardián del artefacto lleva su línea base. Barre solo `src/main`: `src/debug` queda fuera (P-66). - `NoTracesGuardTest`: sin notificaciones, atajos, widgets ni tiles; `FLAG_SECURE` y los envoltorios de ventana propia confinados a `core/security`; `Log`, `println`, `printStackTrace`, `System.out/err`, Crashlytics y Analytics vedados. Barre `src/main` y también `src/debug`. - `ManifestMinimizationTest`, sobre los manifiestos declarados: permisos = inventario aprobado, un único componente propio exportado (el lanzador), FileProvider no exportado con sus dos rutas, `allowBackup=false`, configuración de red sin tráfico en claro, y la retirada de la `PreviewActivity` en `src/debug`. No cubre el manifiesto fusionado del APK —lo que agregan Firebase Auth, GMS y AndroidX—: eso se revisó a mano en P-06, la tabla está en `docs/RESULTADOS_PRUEBAS.md` §3 y los pendientes son P-92 (dos actividades de `firebase-auth`, en las dos variantes) y P-112 (una actividad de `ui-test-manifest`, solo en debug). - `VisualSystemGuardTest`, `SensitiveFieldsGuardTest` y `PaletteContrastTest`.
2. Sobre el artefacto, en la tarea Gradle `verificarCamuflajeDex`, que extrae todas las cadenas del APK de release (DEX, `resources.arsc`, manifiesto, `res/`, `META-INF/`, `assets/` propios) y falla si alguna contiene un término vedado que no esté en la línea base `app/src/test/resources/camuflaje-dex-excepciones.txt`.

La tarea está encadenada a `assembleRelease` y a `bundleRelease`, así que ningún release se construye sin pasar por ella, y se auto-comprueba en cada corrida contra cuatro canarios. También se puede invocar sola —construye lo que necesita:

```
./gradlew verificarCamuflajeDex
# Informe: app/build/reports/camuflaje/dex-release.txt
```

Esto tiene una consecuencia: el comando de verificación habitual (`assembleDebug lint test`) no construye el release y por lo tanto no ejecuta este guardián. Como el guardián solo juzga `outputs/apk/release/app-release.apk`, un APK de debug nunca pasa por él y sale del entorno únicamente con el respaldo de los guardianes de fuentes. Es otro motivo por el que lo que se entregue a una usuaria debe ser release (P-65).

Si el guardián falla, se corrige la causa (renombrar el símbolo o neutralizar el mensaje). Aceptar la cadena en la línea base solo se hace con el motivo escrito al lado y una entrada en `docs/PENDIENTES.md`.

7. Activación de la app

7.1 Cómo funciona

Terminología. La aplicación maneja cuatro credenciales y cada una tiene un solo nombre en todos los textos. `TerminologyGuardTest` falla si un texto usa «clave», «PIN» o «llave», si nombra una contraseña sin decir de cuál se trata, si llama «código» a algo que no sea el código de acceso o el de activación, o si la pantalla de preferencias se llama «Configuración» en lugar de «Ajustes».

Credencial	Qué es	Dónde se escribe
Código de activación	Código único del prototipo; abre el primer uso	Buscador de ciudades
Código de acceso	Lo crea la usuaria; abre el espacio protegido a partir de entonces	Buscador de ciudades
Contraseña de la cuenta	La de la cuenta de Firebase (correo y contraseña); es también la raíz del respaldo de RF-39	Crear cuenta, iniciar sesión, cambiarla, restablecerla y eliminar la cuenta

Credencial	Qué es	Dónde se escribe
Contraseña del archivo PDF	Protege solo un archivo exportado (RF-45)	Pantalla de exportación

Las dos ranuras. El espacio protegido se abre únicamente escribiendo un código en la barra de búsqueda de ciudades. Hay dos ranuras (`CodeSlot`):

1. **`ACTIVATION`.** El código de activación, único del prototipo, que abre el primer uso: bienvenida → consentimiento → cuenta → código de acceso → Cómo usar Aurora.
2. **`USER`.** El código de acceso que crea la propia usuaria (mínimo 6 caracteres y que no sea el nombre de una ciudad real, verificado contra un directorio local de unas 620 entradas en `res/raw/city_index.txt`). Se escribe dos veces, al crearlo y al cambiarlo. Se guarda solo como hash con salt.

`ValidateSecretCodeUseCase` consulta primero la ranura de usuaria: en cuanto existe código de acceso, el de activación deja de abrir. Cualquier texto que no coincida —y cualquier fallo interno— devuelve el mismo resultado que una búsqueda de ciudad sin resultados.

El código de activación solo se siembra en builds debug, por `DebugSecretCodeSeeder` (`app/src/debug/java/.../access/data/`), que lo escribe en el primer arranque si la ranura está vacía. El valor concreto está en ese archivo y no se reproduce aquí. La decisión de que sea un código único entregado en persona está cerrada en P-20 (`docs/DECISIONES_DEL_PROYECTO.md` §1.2).

El primer uso. Las pantallas del flujo viven en el grafo seguro y no navegan por sí mismas: los destinos se deciden en `AuroraApp.kt`.

- **Bienvenida.** El código de activación abre primero una pantalla con el logo y qué es Aurora. No tiene flecha: detrás solo está el clima. Desde ella se pasa al consentimiento.
- **Consentimiento (RF-17, RF-18).** La flecha y el gesto atrás vuelven a la bienvenida. Rechazarlo converge en la salida rápida.
- **Cuenta.** Se crea con correo y contraseña de la cuenta. La flecha y el gesto atrás vuelven al consentimiento. Quien ya tiene cuenta y está en un dispositivo nuevo inicia sesión dentro de este mismo flujo (RF-39); allí la contraseña de la cuenta se pide siempre.

- **Código de acceso.** Se crea con `CompleteCodeSetupUseCase`, que además deja la marca `FROM_START` para que Cómo usar Aurora salga desde su primera pantalla (la marca es de mejor esfuerzo: si no se guarda, la guía no aparece y el resultado no cambia). Con el código creado termina la activación y se navega a `panel`; atrás no regresa al formulario.
- **Cómo usar Aurora.** Descrita abajo.
- **Registro interrumpido (hallazgo A).** Mientras no existe código de acceso, una sesión viva del proveedor o unas credenciales guardadas no abren el paso: vienen de un registro cortado y se descartan, junto con los datos locales que esa cuenta hubiera restaurado (`DiscardUnfinishedActivationUseCase`). El descarte corre al abrir la bienvenida, antes de crear una cuenta y al abrir el inicio de sesión de activación. Ninguna subida a la nube ocurre sin código de acceso. El reingreso con código de acceso ya creado (P-31 C) no pasa por este descarte.
- **Entrada con el código de acceso.** Se muestra unos 700 ms la marca de Aurora, ya dentro del grafo seguro, antes del Inicio.
- **El teclado se cierra al salir de la fachada.** `leaveFacade` (`FacadeExit.kt`) quita el foco, cierra el teclado y espera hasta 500 ms a que baje antes de abrir el espacio privado, tanto con el código de acceso como con el de activación. El tope existe porque en API 26 el alto del teclado puede ser espurio (P-138). La entrada solo ocurre con la app al frente (`repeatOnLifecycle(RESUMED)` y la comprobación `canEnter`): si pasa a segundo plano durante la espera, se pierde, para que nadie vuelva desde recientes y encuentre el interior sin teclear el código (RF-06). Un código que no coincide sigue siendo una búsqueda de ciudad (RF-13).
- **El hash del código corre fuera del hilo principal.** `SecretCodeHasher.hash` es `suspend`; `KeystoreSecretCodeHasher` calcula el HMAC en un carril de un solo hilo (`Dispatchers.IO.limitedParallelism(1)`, experimental en kotlinx-coroutines 1.7.3: el `@OptIn` sale al subir de versión) y la guarda `computeHmac` lanza si corre en el hilo principal, incluso en release. La puerta degrada ese fallo a «búsqueda sin resultados» y la creación del código a `StorageFailure`. Hay una pausa de 400 ms tras la última tecla antes de calcular (la fachada y la práctica de Cómo usar Aurora; en la práctica, «Buscar» evalúa sin esperar la pausa) y la puerta tiene el plazo de la red. `WalkthroughViewModel`, `ChangeCodeViewModel` y `CodeSetupViewModel` registran un limpiador en `SessionManager` que cancela el cálculo en curso dentro de `closeSession()`. La eliminación total (RF-20) borra la llave por el mismo carril. Quedan dos residuales (P-140).

Cómo usar Aurora (RF-10, RF-11). Una sola experiencia enseña a entrar, el espacio interior y la salida: la portada «Conoce Aurora» («Empezar» / «Ahora no»), la parte 1 *Entrar* (práctica del código sobre una réplica de la pantalla de clima, con círculo y flecha dibujados a

mano), la parte 2 *Tu espacio* (Guía, Mi registro, Reconocer tu situación, Resultados guardados, Acciones rápidas y Ajustes) y la parte 3 *Salir* (tocar la nube), más «¡Listo!».

- **Dónde vive.** Como una capa sobre las pestañas dentro de la ruta `panel` del grafo seguro (`WalkthroughHost`/`WalkthroughLayer` en `access/ui`), así que hereda `FLAG_SECURE`, la nube encima de todo y la muerte con el pop. Un solo `WalkthroughViewModel` lleva los diez pasos (`WalkthroughStep`).
- **Controles.** «Atrás», «Siguiente» y «Saltar». «¡Listo!» lleva cuatro recordatorios, el último «Discreta» (D-1 de RF-10). En la parte 3 la acción es la nube y se ofrece «Después». «Siguiente» de la parte 1 permite no practicar (D-2 de RF-10).
- **Numeración.** Las ocho tarjetas se numeran de seguido: la etiqueta es solo «Paso N de 8» (`common_step_label` con `WalkthroughStep.number` y `cardCount`, que salen de las tarjetas y no de un valor fijo). La portada dice «8 pasos cortos. Unos 2 minutos.» (literal; una prueba JVM lo compara con `cardCount`) y nombra Entrar, Tu espacio y Salir sin números.
- **Portada y «¡Listo!».** Son pantallas completas que no se superponen a la app y se arman con piezas del interior (`SoftCard`, `IconListRow`, `V2PrimaryButton`, `CloudLaneStack`/`CloudLaneBar`, `InteriorTypeVoice`): el contenido se desplaza y termina antes de la nube (`reserveCloudLane`) y los botones quedan fijos abajo.
- **Réplica de la fachada en la parte 1.** `WalkthroughTypeIn.kt` pinta, bajo la tarjeta, la pantalla de clima en un marco redondeado con datos fijos de ejemplo y sin red. Para no duplicarla ni hacer que `access` importe `weather` (§2 de ARQUITECTURA), las piezas compartidas viven en `core/ui/v2/FacadeSky.kt`: el cielo (`FacadeSky.brush`), los colores y el teclado de la píldora (`facadeSearchFieldColors`, `facadeSearchKeyboardOptions`), el texto de ayuda, la lupa, el marcador y el bloque de ciudad, temperatura y condición (`FacadeConditionsSummary`); `WeatherScreen` usa las mismas. El `TextField` lo construye cada pantalla dentro de su `SecureTextEntry`, porque la guarda `SensitiveFieldsGuardTest` solo admite campos fuera del kit en esos dos archivos. La réplica no guarda estado: su barra escribe en `WalkthroughUiState`. «Buscar» llama a `WalkthroughViewModel.onPracticeSubmit`, que cancela la pausa y evalúa en el acto con la misma puerta; el aviso «Ese no es tu código de acceso…» es una región viva que se trae a la vista con `BringIntoViewRequester`. `HandDrawnCue` admite un `halo` (trazo blanco más ancho debajo) para que el círculo se lea sobre el cielo oscuro y la flecha sobre el lavanda; sin el borde de la tarjeta a la vista, solo dibuja el círculo.
- **Marca y reanudación.** `SealedWalkthroughRepository` guarda en `view_meta` (preferencias neutras, sellado con la DEK de la cuenta, 16 bytes, claves `v0`/`v1`) una marca con dos valores: `FROM_START` («desde el inicio», la deja `CompleteCodeSetupUseCase` al crear el código; se consume antes de abrir) y

`TO_FINISH` («por cerrar», se arma al llegar al paso de la nube y se desarma si se deja sin salir). En la siguiente entrada la guía muestra «¡Listo!» una vez. Como la nube cierra también la sesión del proveedor y sin sesión no hay DEK, el ViewModel consulta `PayloadCipher.isReady()` cada 250 ms hasta 10 s antes de abrir la marca; si el plazo vence (sin red), la marca queda para la siguiente entrada. Muere con la eliminación de la cuenta y con el descarte de una activación sin terminar (`WalkthroughEraser`).

- **Estilo propio.** Paleta lavanda aparte de `V2Palette` (`GuidePalette`), tarjeta y botón (`GuideCard`, `GuidePrimaryButton`), trazos a mano (`HandDrawn.kt`, con `Path` y `PathMeasure`, sin imágenes) y la fuente Patrick Hand (OFL; el archivo `.ttf` pesa ~210 KB y su licencia está en `docs/licencias/`). El hueco que ilumina (`Spotlight.kt`) se desliza y la tarjeta nunca tapa lo iluminado. Este estilo queda solo en las tarjetas. Con «Quitar animaciones» todo queda completo y quieto.
- **Ajustes.** Una sola fila, «Ver cómo usar Aurora»; deja una bandera neutra en `panel` y la guía arranca en «Conoce Aurora».

«Seguridad en Aurora» (RF-11). Fila en Ajustes › Tu acceso que abre `access/ui/SafetyNotesScreen.kt` (ruta `settings_notes` del grafo seguro): pantalla estática, sin ViewModel, con el esquema de `PrivacyScreen` (`V2BackHeader`, `SoftCard` con `IconListRow`, filas como `IconNote` en `IconNoteCard`, que comparte con «Conoce Aurora» y «¡Listo!»). Sus frases describen comportamientos del código: `SessionManager` (5 min de inactividad, gracia de ida y vuelta al sistema), `SecureNavigation` (`FLAG_SECURE`), `ValidateSecretCodeUseCase`, `GuidanceViewModel` (nada se guarda a mitad), SQLCipher y `KeystoreMediaCipher`, `PayloadCipher`/`RecoverableMediaCipher` con la DEK envuelta por la contraseña, `firestore.rules` (`/users` solo para su dueña), `ImageImporter` (re-codifica sin metadatos), `CaptureBuffer` y `DeleteAccountUseCase` (exige conexión). Si uno de esos comportamientos cambia, hay que cambiar el texto; `SafetyNotesTextTest` ata los «5 minutos» a `SessionManager`. Prueba instrumentada: `SafetyNotesScreenTest`.

Comportamientos transversales.

- **Salida rápida.** La nube queda sobre el teclado y sale con un solo toque también con una hoja, un diálogo o el selector de fecha abiertos.

- **Salir con cambios sin guardar.** El editor de registros y el cuestionario preguntan antes; la nube y el cierre por inactividad nunca preguntan.
- **Adjuntos.** Antes del selector del sistema aparece una hoja que explica cómo volver sin elegir nada; cada adjunto se muestra con un nombre genérico («Imagen 1 · 12 oct»); cada archivo se verifica después de sellarlo (§4.2).
- **Fachada.** La búsqueda y el clima muestran carga, y ante un fallo o tras 10 s, un error con «Reintentar».
- **Cuestionario.** Toda la tarjeta lo abre y «Empezar» va dentro.
- **Franja de la nube.** En Inicio y en la portada de la Guía la zona que se desplaza termina antes de la nube (`reserveCloudLane`), con cualquier letra.
- **Letra grande.** Las tres Acciones rápidas se apilan en lugar de reducir la letra; las etiquetas de pestaña van en una línea.
- **Selector de fecha.** Su contenido aparta el teclado igual que la nube (`imePadding()`): en API 26 la plataforma reporta a veces un inset de teclado sin teclado y la nube subiría sobre «Aceptar» (P-138).
- **Guarda de camuflaje.** `CamouflageResourcesTest` busca los términos de tres letras como palabra entera, pero solo en el texto de `<string>` e `<item>`; nombres de archivo, identificadores y atributos se buscan como subcadena (el texto «3 pasos» contenía «sos»).

Consecuencias prácticas:

- Para volver a hacer un primer uso completo hay que dejar la aplicación como recién instalada (desinstalar y reinstalar). Mientras exista código de acceso, el de activación no sirve. Procedimiento en `docs/PREPARACION_SESIONES.md`.
- Si una usuaria olvida su código de acceso, la vía aprobada (P-18, opción A) es reinstalar o borrar datos → entrar con el código de activación → iniciar sesión con la cuenta existente → crear un código de acceso nuevo. Los registros vuelven desde la nube. No se documenta dentro de la aplicación, para no dejar rastro.
- En release, tras una eliminación total de cuenta el espacio protegido queda inaccesible en esa instalación, porque la resiembra solo existe en debug (P-55).

7.2 La limitación de P-111

El primer arranque del release en instalación limpia no se ha ejercitado nunca. Como ninguna build de release siembra la clave de activación —P-20 decidió la clave única entregada en persona, pero esa decisión no tiene implementación en `app/src/release/`, que no existe—, la única forma de probar el release es instalar el debug, activar y actualizar in-place. Eso deja probada la lectura de la base SQLCipher y del Keystore bajo R8 (la clave personal abre, y el registro bajó de la nube y se descifró), pero sin probar la creación: la primera apertura de `journal_store.db` / `guidance_store.db`, el alta de los alias del Keystore, las

migraciones de Room y el alta de cuenta, todo bajo ofuscación. Es donde fallaría una regla `keep` incompleta de la reflexión de Room, y es lo que hará el teléfono del piloto, que recibirá un release limpio. Cómo cerrarlo (es la misma tarea que resuelve la carencia de P-20): dar al release una vía de activación (un sembrador propio, la clave en `BuildConfig` o la entrega remota por Firestore) y entonces correr el bloque A entero sobre una instalación limpia de release. Mientras no exista esa vía, la limitación no puede cerrarse.

(Resume P-111 de `docs/PENDIENTES.md`.)

Mientras P-111 siga abierto, cualquier prueba que necesite un primer uso completo —evaluación heurística, sesiones de usabilidad— tiene que hacerse con la build debug. El costo de esa elección y sus implicaciones están en `docs/PREPARACION_SESIONES.md`.

8. Panel web

Panel interno para gestionar el contenido que consume la aplicación: tipos (RF-41), rutas/entidades/líneas (RF-42), cuestionario (RF-43) y la publicación reflejada en la app (RF-44). Autentica a la administradora (RF-40) y nunca accede a datos de usuarias (RNF-06). Vive en `admin/`, con Vite + React + TypeScript estricto y el SDK web de Firebase, sin backend propio.

8.1 Instalación y variables de entorno

```
cd admin
npm install
cp .env.example .env
```

Rellenar en `.env` las seis variables `VITE_FIREBASE_*` con los valores de *Consola → Configuración del proyecto → Tus apps → App web → SDK*. Si no hay app web en el proyecto, crearla con el icono `</>`; eso no altera el APK Android.

Esos valores no son secretos: la configuración web de Firebase viaja en cualquier bundle de cliente, igual que ya viaja en el `google-services.json` del APK. La seguridad real la dan `firestore.rules` y el custom claim `admin`. Se usan variables de entorno por higiene, y `.env` está vedado por `.gitignore`.

8.2 Ejecutar, probar y construir

```
npm run dev      # servidor de desarrollo en http://localhost:5173
npm run test     # Vitest: validaciones críticas (teléfonos, coherencia del
cuestionario)
npm run lint     # ESLint + prettier --check
npm run format   # prettier --write
npm run build    # tsc -b && vite build → dist/
```

Para entrar hace falta la cuenta creada en §5.5. Una cuenta sin el claim `admin` se autentica, el panel detecta que falta el rol, cierra la sesión de inmediato y muestra el mismo mensaje genérico que unas credenciales inválidas: no revela si la cuenta existe.

8.3 Despliegue

`npm run build` genera `dist/`, estático, que puede servirse en cualquier hosting. No se ha desplegado —queda fuera del alcance de la Fase 3— y hay una condición si se hace: el panel no debe servirse desde el mismo dominio ni con una identidad visible que sugiera relación con la aplicación de clima.

8.4 Arquitectura del panel

```
admin/src/
├── firebase/config.ts     Init de Firebase (Auth + Firestore) desde .env
├── auth/                  RF-40: AuthProvider (único estado global) + login
├── lib/
│   ├── drafts.ts          Flujo borrador → publicado (colección /drafts)
│   ├── useEntries.ts      Carga de vivo + borrador por recurso
│   ├── useResourceEditor.ts Máquina de estados del editor
│   ├── ids.ts             Ids opacos (t01/r01/a01/l01…)
│   ├── phone.ts           Validación de teléfonos colombianos (RF-42)
│   ├── validation.ts      Marcador [POR CONFIRMAR] y tipos
│   ├── parse.ts           Lectura tolerante (espejo de los DTO de la app)
│   └── contentVisuals.ts  Espejo exacto de la resolución visual de la app
├── components/            Presentacionales
└── resources/
    ├── content/           RF-41: tipos + vista previa
    ├── routes/            RF-42: rutas, entidades y líneas en pestañas
    ├── questionnaire/     RF-43: esquema del formulario + coherencia
    └── publish/           RF-44: estado global y publicación pendiente
```

`contentVisuals.ts` y `core/ui/v2/ContentVisual.kt` de la app son pareja declarada: replican la misma resolución de icono y acento y se modifican juntos.

9. Contenido

El contenido de la guía y el cuestionario viven en Firestore, no en el APK. Hay dos vías para escribirlos: la siembra por script y el panel.

9.1 Modelo de datos

Colección	Documento	Esquema	Campos
`topics`	tipo de violencia (también las opciones de tipo del registro de la usuaria)	2	`schemaVersion, order, icon, accent, title, summary, signals[], examples[]`
`routes`	sector de la ruta (Salud / Protección / Justicia)	2	`schemaVersion, order, icon, accent, title, summary, agencyIds[]`
`agencies`	entidad de atención	1	`schemaVersion, name, role, whenToUse, phones[], address?, hours?`
`lines`	línea de atención	1	`schemaVersion, order, name, number, description, hours?`
`forms`	configuración completa del cuestionario (un único documento)	2	`schemaVersion, version, title, intro, notice, scales[], blocks[], questions[], dimensions[], recognition, thresholdFlags[], countFlags[], municipalities[], levels[]`

Los nombres de colección y los ids de documento son neutros y opacos por diseño: viajan en las rutas de la API y quedan en la caché local del dispositivo. Cada documento lleva `schemaVersion` y la aplicación ignora las versiones que no conoce, de modo que el panel puede evolucionar el esquema sin romper instalaciones viejas.

`icon` y `accent` no son datos libres: solo seleccionan dentro de una biblioteca cerrada de la aplicación (10 iconos y 6 acentos), nombrada por el dibujo y el matiz y nunca por el significado. Un valor ausente, de tipo inesperado o fuera de la biblioteca cae al mapeo automático por id, campo por campo.

> Nota: `tools/seed/README.md` describe `forms` con el esquema 1 y una sola escala. Ese texto no refleja el instrumento vigente, la versión vigente de 12 preguntas (ADR del cuestionario); la fuente de verdad del esquema vigente es `tools/seed/content/forms.json` y el ADR.

El cuestionario vive en un solo documento para que publicar sea atómico: la aplicación descarta por completo cualquier documento incompleto, de modo que nunca pueden regir preguntas nuevas con niveles viejos.

9.2 Siembra con tools/seed

> **Precaución.** Los scripts de `tools/seed` escriben en el proyecto de Firebase al que pertenece la llave de servicio, que puede ser el de producción. Antes de correrlos se confirma el `project_id` de la llave, y si el script tiene modo de prueba, se ejecuta primero en ese modo.
>
> Antes de la primera siembra, dos condiciones obligatorias:

1. `firestore.rules` desplegado (§5.3). Mientras el proyecto siga en modo de prueba, el contenido es legible y escribible por cualquiera. El script pide confirmación de este paso; se puede saltar con `AURORA_RULES_PUBLISHED=1`.
2. Revisar qué documentos siguen con el marcador `[POR CONFIRMAR]`: el script los omite por omisión. `--borrador` los siembra de todas formas, y solo debe usarse en un entorno de prueba.

```
cd tools/seed
npm ci     # instala exactamente las versiones del package-lock.json
export GOOGLE_APPLICATION_CREDENTIALS="$HOME/.config/aurora-seed/service-
account.json"
npm run seed
# en entorno de prueba, para incluir lo marcado:
npm run seed -- --borrador
```

- Es idempotente (`set` con `merge`): tras cada corrección se edita el JSON y se vuelve a ejecutar.
- No borra documentos que se retiren de los JSON: hay que eliminarlos a mano en la consola.
- Usar `npm ci` y no `npm install`: el `package-lock.json` versionado fija las versiones exactas en la máquina que custodia la llave administrativa.

9.3 Publicación acotada de tipos (publicar-tipos)

Escribe únicamente los ids que se le indiquen en `topics`, sin tocar el resto. Por omisión corre en modo de prueba: lee el JSON, muestra qué escribiría y termina sin conectarse a Firestore.

```
cd tools/seed
export GOOGLE_APPLICATION_CREDENTIALS="$HOME/.config/aurora-seed/service-
account.json"
```

```
npm run publicar-tipos                       # modo de prueba
npm run publicar-tipos -- --publicar         # escribe de verdad (pide
confirmación)
npm run publicar-tipos -- --ids t05,t06      # lista explícita; nunca «todos»
```

- Escribe el documento vivo y su borrador del panel, para que el panel no quede con una versión congelada que al publicar sobrescribiría lo validado.
- Requisito de orden: el APK con soporte del esquema vigente debe estar instalado antes de publicar documentos de un esquema nuevo; un APK anterior los descarta.
- Si se invoca con `stdin` por tubería se cuelga en la segunda confirmación. Hay salida: `export AURORA_RULES_PUBLISHED=1` y una sola respuesta en la tubería.

9.4 Edición desde el panel: borradores y publicación

La aplicación lee las colecciones vivas completas y sin filtros, así que los borradores viven en una colección aparte, `drafts`, que solo el rol `admin` puede leer y que la aplicación nunca consulta.

- Guardar borrador es libre y asciende el documento al esquema vigente.
- Publicar copia el campo `data` del borrador *tal cual* al documento vivo, y exige pasar la validación dura del recurso: teléfonos bien formados, referencias resolubles, iconos y acentos dentro de la biblioteca cerrada, pesos y umbrales coherentes, y ningún `[POR CONFIRMAR]`.
- El panel bloquea eliminar una entidad referenciada por una ruta, y publicar una ruta antes que las entidades que cita.
- En el cuestionario, la versión sube automáticamente al publicar; la aplicación toma siempre el documento de mayor `version`.

El camino de verificación completo panel → app está en `admin/README.md` §6.

10. Pruebas

10.1 Comandos

> **Precaución.** Las pruebas instrumentadas instalan y desinstalan la aplicación. Sin la opción `-Pandroid.injected.androidTest.leaveApksInstalledAfterRun=true` borran todos los datos locales del dispositivo, incluidas las llaves del Keystore (§13.2). No se corren en un teléfono con una cuenta en uso.

```
# Unitarias de JVM (debug y release), estilo, lint y los dos APK:
./gradlew cleanTestDebugUnitTest cleanTestReleaseUnitTest test ktlintCheck lint
assembleDebug assembleRelease
```

```
# Por partes:
./gradlew test                       # unitarias de JVM
./gradlew ktlintCheck                # estilo de Kotlin
./gradlew lint                       # Android Lint
./gradlew assembleDebug              # compilar debug
./gradlew assembleRelease            # release + guardián del camuflaje del APK

# Instrumentadas (requieren emulador o dispositivo):
./gradlew connectedDebugAndroidTest -
Pandroid.injected.androidTest.leaveApksInstalledAfterRun=true

# Panel web:
cd admin && npm run test && npm run lint && npm run build

# Reglas de Firestore (requiere Firebase CLI con el emulador):
firebase emulators:exec --only firestore "node --test firestore/rules.test.mjs"
```

El `clean…` explícito antes de `test` evita que Gradle reutilice resultados en caché y dé por buena una corrida que no ocurrió. El parámetro `leaveApksInstalledAfterRun` no es opcional si hay datos que conservar: ver §13.2.

Las pruebas unitarias corren dos veces, una por variante (debug y release), porque los guardianes de camuflaje y de recursos deben valer en ambas.

10.2 Dónde están el plan y los resultados

- `docs/PLAN_PRUEBAS.md` — los casos CP-01 a CP-39 con su tipo (unitaria, instrumentada, caja negra, seguridad, usabilidad), el requisito que verifican y la fase en que corren.
- `docs/RESULTADOS_PRUEBAS.md` — resultados de la corrida automatizada (§1), la verificación de R8 sobre el release (§2), el manifiesto fusionado y el tamaño del APK (§3), la matriz de cobertura de los CP (§4), las pasadas manuales ya ejecutadas (§5) y la lista única de corridas manuales con su resultado (§8).
- `docs/INFORME_SEGURIDAD_OWASP.md` — la evaluación MASVS/MASTG sobre el APK de release, con las evidencias en `docs/evidencias-owasp/`.

Cifras vigentes de `v0.3.4-trl4`: 1068 pruebas de JVM por variante en 132 clases, sin fallos; 199 instrumentadas en 39 clases, sin fallos en API 26 y API 30, con una omitida en API 26 —la del gesto de atrás de `AttachPickerCancelTest`, que no existe antes de Android 10 —; ktlint sin violaciones; Android Lint con 0 errores; el panel con 91 pruebas en verde. Las instrumentadas se corren en API 26 y en API 30: varias pruebas del teclado, del selector de

fecha y del cifrado de adjuntos solo muestran su caso en Android 8. La cifra exacta de cada corrida está fechada en `docs/RESULTADOS_PRUEBAS.md` y es la referencia, no este párrafo.

10.3 Lo que no se puede automatizar

Tres bloques siguen siendo manuales y están documentados paso a paso en la §8 de `docs/RESULTADOS_PRUEBAS.md`:

- Las reglas de Firestore, mientras no haya emulador instalado: se verifican en el *Rules playground* con la tabla de `firestore/README.md` (CP-37).
- La regresión visual contra los mockups de `docs/mockups-v2/` y el contraste de los controles.
- Los flujos que cruzan procesos del sistema: cámara, selector de documentos, marcador telefónico, miniatura de recientes, fuente al 200 %, modo avión y segundo dispositivo.

11. Mantenimiento

Todos los scripts de esta sección necesitan la llave de servicio exportada (§5.4), corren en modo de prueba por omisión y escriben solo cuando se les pasa la bandera explícita.

11.1 Inventariar cuentas

Solo lectura. No toca Firestore ni lee el contenido de ningún documento:

```
cd tools/admin
export GOOGLE_APPLICATION_CREDENTIALS="$HOME/.config/aurora-seed/service-
account.json"
node listar-cuentas.mjs
```

Imprime correo, uid, fecha de creación y último acceso de cada cuenta, ordenadas por antigüedad. Es la forma de decidir qué cuenta sobra.

11.2 Eliminar una cuenta de prueba y sus datos

Eliminar un usuario de Auth no borra sus documentos de Firestore, así que el script purga primero `users/{uid}` —recursivo: `entries`, `media`, `mediaparts`, `meta`, `outcomes` — y después elimina la cuenta, verificando al final que ya no existe. Nunca lee ni imprime el contenido de los documentos: solo los cuenta.

Precaución. Con `--delete` el borrado es permanente: los registros, los adjuntos y la cuenta no se pueden recuperar. Se usa solo con cuentas de prueba, después de revisar el resultado del modo de inspección.

```
cd tools/admin
export GOOGLE_APPLICATION_CREDENTIALS="$HOME/.config/aurora-seed/service-
account.json"
node delete-test-account.mjs --email <correo>              # solo inspección
node delete-test-account.mjs --email <correo> --delete      # purga + elimina +
verifica
```

11.3 Limpiar borradores obsoletos del panel

Cuando se publica contenido por `seed.mjs` o `publicar-tipos`, el borrador del panel se queda congelado en lo último que se tocó allí. Este script detecta los borradores cuyo contenido ya no coincide con el documento vivo:

Precaución. Con `--borrar` los borradores se eliminan de Firestore. El script guarda antes una copia (§11.4), pero conviene revisar primero la lista del modo de prueba.

```
cd tools/admin
export GOOGLE_APPLICATION_CREDENTIALS="$HOME/.config/aurora-seed/service-
account.json"
node limpiar-borradores.mjs                    # modo de prueba: lista borrador
vs. publicado
node limpiar-borradores.mjs --ids topics__t05  # acotar a ids concretos
node limpiar-borradores.mjs --borrar           # borra de verdad (pide
confirmación)
```

11.4 Respaldos

- Antes de borrar un borrador, el propio script guarda una copia íntegra en `~/.config/aurora-seed/respaldos/borradores-obsoletos-<fecha>.json`, también en modo de prueba. Va fuera del repositorio porque lleva el contenido de los documentos; `--respaldo <ruta>` cambia el destino y el script rechaza escribir dentro del repositorio.
- El contenido publicado tiene su fuente en el repositorio: `tools/seed/content/~.json` reproduce lo que debe haber en Firestore, y `tools/seed/source/~.md` guarda las fuentes de las que salieron esos JSON. Reponer el contenido es volver a sembrar.
- Los datos de las usuarias no se respaldan desde la operación, por diseño: lo que hay en Firestore son blobs que solo se descifran en el dispositivo con la contraseña de la cuenta. No existe —ni debe existir— una vía administrativa para leerlos o exportarlos.
- Firestore no tiene exportación automática en plan Spark. Si se quisiera una copia de las colecciones de contenido, habría que hacerla a mano o subir de plan.

12. Seguridad operativa

12.1 Reglas

`firestore.rules` es la frontera entre el contenido administrable y los datos de las usuarias, y es la implementación de RNF-06. Tres reglas de operación:

- Se despliega antes de la primera siembra y después de cada cambio. Un cambio en el archivo no surte efecto hasta publicarlo.
- Después de cada despliegue se verifica al menos el caso del claim `admin` intentando leer `users/<uid>/...`, que debe quedar denegado (tabla en `firestore/README.md`).
- El claim `admin` no se otorga nunca desde el panel ni desde la consola a mano: solo con `create-admin.mjs`.

12.2 Nada en logs

Ninguna información del espacio protegido se registra. En la práctica:

- No hay `Log.*` ni `println` con datos del espacio protegido, y en release las llamadas a `android.util.Log` se eliminan por `-assumenosideeffects`.
- No hay Crashlytics, Analytics ni ninguna biblioteca de analítica de terceros, y no debe agregarse ninguna.
- Los mensajes de error visibles son genéricos: un fallo de red o de Firestore nunca llega a la pantalla con su detalle.
- Los scripts de `tools/` cuentan documentos pero no imprimen su contenido, ni secretos, ni contraseñas.

En builds debug esto no se cumple del todo: la biblioteca de PDF escribe en logcat trazas técnicas sobre fuentes y estructura de los documentos anexados (nunca contenido ni contraseñas), porque la regla que las silencia solo actúa con R8. Es uno de los motivos por los que el piloto debe hacerse con release (P-65).

12.3 Secretos fuera del repositorio

`.gitignore` veda lo siguiente, y debe verificarse tras cualquier reorganización:

Qué	Dónde debe vivir
`google-services.json`	`app/`, sin versionar
Keystores de firma (`~.jks`, `~.keystore`)	Fuera del repositorio
Llave de servicio (`service-account*.json`, `~-adminsdk-~.json`)	`~/.config/aurora-seed/`
Copias de borradores (`borradores-obsoletos-*.json`)	`~/.config/aurora-seed/respaldos/`
`.env` del panel	`admin/.env`, sin versionar
`local.properties`	Sin versionar
Capturas y PDF exportados de las corridas manuales	Fuera del repositorio

Tres barreras protegen la llave de servicio: la ruta canónica fuera del árbol, `.gitignore` y la comprobación que cada script hace al arrancar, que aborta si la ruta cae dentro del repositorio.

La configuración web de Firebase (la clave de API del `google-services.json` y las variables `VITE_FIREBASE_*`) no es un secreto: viaja en cualquier cliente. Lo que protege el proyecto son las reglas y el claim, no el ocultamiento de esos valores.

12.4 Registro de cuentas (P-109)

El registro por correo y contraseña de Firebase Auth está abierto por omisión, y el código de activación cierra la puerta de la interfaz, pero no la de la API. Quien obtenga la clave web —que viaja en cualquier APK, como en toda aplicación de Firebase— puede crear cuentas directamente y leer el contenido de orientación, que es información de interés público (rutas y líneas de atención, no datos de usuarias). El daño posible es consumo de cuota, cuentas basura y, en un proyecto de pago, costo.

Qué hacer:

- Mantener el registro activo solo mientras se necesite (por ejemplo, durante las sesiones de usabilidad, en las que las participantes crean cuenta) y desactivarlo al terminar en *Authentication → Settings → User actions*. Es un interruptor y se vuelve a activar cuando haga falta.
- Confirmar que el proyecto sigue en plan Spark, para que ningún abuso pueda convertirse en una factura.

- Antes del piloto: activar App Check con Play Integrity, que ata las peticiones a la aplicación auténtica y es la mitigación de fondo.
- Pasar `listar-cuentas.mjs` periódicamente y eliminar con `delete-test-account.mjs` lo que sobre (§11).

13. Problemas frecuentes y cómo resolverlos

13.1 Gradle falla con «What went wrong: 27»

Gradle 8.10.2 no arranca con el JDK 27. El síntoma induce a error: si reutiliza un demonio vivo parece funcionar, y falla en cuanto necesita uno nuevo. Solución:

```
JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"
./gradlew <tarea>
```

13.2 connectedDebugAndroidTest borró los datos del dispositivo

La tarea instala el APK y el de pruebas, corre las pruebas y desinstala ambos. En un dispositivo con una cuenta viva eso borra el estado local completo: bases SQLCipher, adjuntos sellados de `files/media_store`, `shared_prefs` y los alias del Android Keystore. La nube no se toca.

- **Evitarlo:** `-Pandroid.injected.androidTest.leaveApksInstalledAfterRun=true`, o correr las instrumentadas en un segundo emulador.
- **Recuperarse:** `./gradlew installDebug`, entrar con el código de activación (el sembrador de debug lo repone al estar vacía la ranura) → bienvenida → consentimiento → «¿Ya tienes una cuenta? Inicia sesión.» → correo y contraseña de la cuenta. La recuperación multi-dispositivo repone los registros y los adjuntos desde la nube. Hay que crear un código de acceso nuevo: el anterior se perdió con el Keystore.

13.3 Un cambio de interfaz «no aparece» en el dispositivo

`assembleDebug` compila pero no instala. Instalar y comprobar la hora antes de sacar conclusiones:

```
adb install -r app/build/outputs/apk/debug/app-debug.apk
adb shell dumpsys package com.auroraclima.app | grep lastUpdateTime
```

13.4 La compilación se detiene en google-services

Falta `app/google-services.json`, o su nombre de paquete no coincide con el `applicationId` (`com.auroraclima.app`). Ver §6.2.

13.5 El release falla en verificarCamuflajeDex

El guardián encontró vocabulario propio en el artefacto. El informe `app/build/reports/camuflaje/dex-release.txt` dice qué término y en qué cadena. Se debe corregir la causa (renombrar el símbolo, neutralizar el mensaje) y no ampliar la línea base. Ver §6.5.

El guardián también falla —por diseño— si aparece en el APK una familia de entradas que nadie juzga y que nadie declaró fuera de alcance. En ese caso hay que decidir explícitamente si entra en el alcance o se excluye con el motivo escrito.

13.6 publicar-tipos o limpiar-borradores se quedan colgados

Hacen dos preguntas con una sola interfaz de lectura. Si `stdin` llega por tubería, la primera respuesta se consume y la segunda nunca se resuelve. Salida:

```
export AURORA_RULES_PUBLISHED=1
echo s | npm run --silent publicar-tipos -- --publicar
```

13.7 El script aborta diciendo que la llave está dentro del repositorio

Es el comportamiento esperado de la barrera B-1. Mover la llave a `~/.config/aurora-seed/` y volver a exportar `GOOGLE_APPLICATION_CREDENTIALS`.

13.8 El panel autentica pero no entra

A la cuenta le falta el claim `admin`, o el token es anterior a su asignación. Verificar el claim (§5.5) y volver a iniciar sesión. El mensaje es el mismo que el de credenciales inválidas por diseño: no revela si la cuenta existe.

13.9 El contenido nuevo no aparece en la app

Tres causas, en orden de probabilidad:

1. Se guardó borrador pero no se publicó. La aplicación lee solo las colecciones vivas.
2. El documento tiene un `schemaVersion` que el APK instalado no conoce; la aplicación lo descarta. Actualizar el APK antes de publicar documentos de un esquema nuevo.
3. El documento está incompleto o tiene una referencia sin resolver. La aplicación omite documentos incompletos, y en el caso del cuestionario descarta el documento entero.

13.10 El emulador entra en ANR y la app no arranca

Un AVD con 1 núcleo y 2 GB no sostiene el sistema y la aplicación a la vez. Relanzarlo con más recursos:

```
emulator -avd <nombre-del-avd> -cores 4 -memory 4096
```

La medición válida de arranque se hizo sobre un AVD Pixel 2 / API 30 con 4 núcleos y 2 GB: 265 / 228 / 231 ms (ver P-80).

13.11 No se puede capturar una pantalla del espacio protegido

Es el efecto esperado de `FLAG_SECURE`: `adb shell screencap` devuelve 0 bytes. En emulador —y solo en emulador— el framebuffer del host sí se puede leer:

```
adb emu screenrecord screenshot <directorio>
```

Al usar esas imágenes como evidencia debe indicarse que es una capacidad del emulador, no una vía disponible en un teléfono.

13.12 Al automatizar por adb, el selector de documentos no responde

`DocumentsUI` no atiende toques sintéticos en sus filas (sí en los botones de la barra). Si hace falta elegir un archivo, la vía que funciona es la cámara. Además, `input text` escribe donde está el foco, no donde se toca. Patrón que funciona: tocar el campo → escribir → `input keyevent 4` para cerrar el teclado → volcar el árbol → tocar el siguiente campo con sus coordenadas nuevas.

13.13 Las instrumentadas fallan «al azar» en el emulador de API 26

La imagen de API 26 acumula ANR de *System UI* y de Google Play Services, sobre todo con dos emuladores encendidos o tras varios reinicios. Su diálogo («System UI isn't responding») queda encima de todo e intercepta la tecla atrás, así que pruebas como `AttachPickerCancelTest` o `SecureWindowCloudTest` fallan sin relación con la app. Además, `adb` puede quedar colgado (`adb shell` no responde y Gradle informa «0 tests» o «No compatible devices»).

- Comprobarlo: `adb logcat -d -b events | grep am_anr` y `adb shell dumpsys window | grep mCurrentFocus` (si el foco es «Application Not Responding», es esto).
- Resolverlo: un emulador a la vez, arranque en frío (`emulator -avd <nombre> -no-snapshot`), esperar a que se asiente antes de correr, y si `adb` no responde, `adb kill-server`.
- Si el equipo anfitrión está cargado, arrancar el AVD sin ventana (`emulator -avd <nombre> -no-snapshot -no-window`): con la ventana, la suite completa se proyectaba en más de 90 minutos y sin ella tardó menos de dos minutos. Las pruebas no necesitan ver la pantalla. Con API 30 pasa lo mismo: con ventana, una prueba cada ~30 s y fallos de «el teclado no llegó a abrirse»; sin ventana, 181 pruebas en 2 min 29 s y ninguna falla.
- En API 26 la prueba del gesto de atrás de `AttachPickerCancelTest` se omite deliberadamente: el gesto llegó con Android 10, y antes deslizar desde el borde abre el cajón del selector de documentos.

14. Pendientes antes de un piloto

Lista de lo que hay que resolver antes de poner la aplicación en manos de personas. El detalle de cada punto está en `docs/PENDIENTES.md` con su número.

14.1 Bloqueantes

#	Qué	Quién
P-108	El camuflaje de este prototipo ya no cumple su función. El repositorio es público, y con él son públicos el nombre, el ícono, el nombre de paquete, la apariencia de la fachada, el mecanismo de entrada y la clave de activación del debug. Un despliegue con usuarias reales no puede reutilizar nada de esa identidad: hay que cambiar nombre, ícono, nombre de paquete, apariencia y clave de activación, en un repositorio nuevo y privado desde el primer commit, porque el historial de este no se puede reescribir. Sin esto, RNF-05 no se cumple	Decisión y nueva implementación
P-19 / P-88	Personalizar la plantilla del correo de recuperación en la consola de Firebase y el *Public-facing name*, y verificar con un correo real que no aparece ningún término ajeno a la identidad de clima. No entregar un APK con la pantalla de recuperación antes de hacerlo	Manual (consola)
P-91	Firmar el release con un keystore propio fuera del repositorio, con los esquemas v2 y v3. Hoy se firma con el certificado de depuración, cuyo almacén usa una contraseña universalmente conocida	Manual + cambio en `build.gradle.kts`
P-111	Dar al release una vía de activación (sembrador propio, `BuildConfig` o entrega remota) y correr el bloque A completo sobre una instalación limpia de release. Hoy la creación de las bases cifradas, los alias del Keystore y las migraciones de Room nunca se han ejercitado bajo R8	Decisión + sesión de implementación

14.2 Importantes

#	Qué
P-109	Desactivar el registro de cuentas al terminar las sesiones de evaluación; confirmar plan Spark; activar App Check con Play Integrity antes del piloto
P-92	Evaluar retirar con `tools:node="remove"` las dos actividades exportadas que `firebase-auth` fusiona en el manifiesto (`genericidp://` y `recaptcha://`), que Aurora no usa, y verificar después que login, creación de cuenta y recuperación siguen funcionando: Firebase puede usar reCAPTCHA como defensa antiabuso
P-139	Corrida manual de Cómo usar Aurora (orden, salida real y reanudación, Ajustes, letra grande, TalkBack, recientes, teclado sobre Inicio y selector de fecha en API 26) en el teléfono y en `Aurora_API26`
P-140	«Eliminar todo» y la llave del código: una consulta de la puerta en el instante del borrado puede volver a crear la llave (se cierra haciendo que solo la creación del código la genere), y el borrado de la llave puede seguir aunque se cancele (`runCatching` atrapa la `CancellationException` en `AccessCredentialsEraser`)
P-118	Ligar el dispositivo a la cuenta del primer inicio de sesión con código. Hoy, si alguien cierra sesión y entra con otra cuenta en el mismo teléfono, los registros locales de la primera pueden subir a la nube de la segunda. No afecta a las sesiones de evaluación: teléfonos propios, una cuenta de prueba por participante y reinstalación entre sesiones. Cambia el reingreso, por eso se dejó para después de ellas

#	Qué
P-112	Caso análogo a P-92, solo en debug: retirar la `androidx.activity.ComponentActivity` exportada que fusiona `ui-test-manifest`, o mover esa dependencia a `androidTestImplementation`, y fijar la retirada en `ManifestMinimizationTest`. No se ve en pantalla y no afecta a las sesiones de evaluación, pero es superficie innecesaria en el artefacto que se instala en los teléfonos
P-131	Repetir en un teléfono con Android 8.0 u 8.1, y en el del piloto, la prueba y la medición de adjuntos grandes. El defecto del Keystore de esas versiones y su corrección solo se midieron en emulador; guardar 10 MB con verificación tardó 5–6,6 s en el emulador de API 30 con el computador cargado
P-80	Medir el arranque en frío en el teléfono del piloto (`adb shell am start -W`, TotalTime) contra el techo de 3 s de RNF-07. En emulador da 265 ms, pero un emulador ARM sobre Apple Silicon es mejor que un gama media real. Queda además la deuda conocida: `MainActivity` y la raíz de la app construyen Firebase en el hilo principal, y el refactor sugerido es diferirlos con `Provider`
P-107	Repetir la verificación de que el teclado no aprende ni sugiere con el teclado de fábrica de los teléfonos del piloto. Lo medido solo demuestra que Gboard honra las banderas; honrarlas es decisión de cada teclado
P-65	Decidir formalmente que el piloto se hace solo con release: en debug los identificadores semánticos de la interfaz quedan legibles en el DEX y la biblioteca de PDF escribe trazas en logcat. Nota: hoy esto choca con P-111, que impide probar un primer uso completo en release
P-55	Pendiente de revisión en la reunión de seguimiento: ratificar que, en release, tras una eliminación total de cuenta el espacio protegido queda inaccesible en esa instalación y volver a usar la aplicación exige reinstalar
P-84	Residual sin cierre posible: entre API 26 y 32 el código a medio teclear puede quedar en la miniatura de recientes, porque la API que lo evita existe desde API 33 y el `minSdk` del proyecto es 26
P-95, P-105	Residuales declarados de la protección de los campos sensibles: las acciones semánticas de accesibilidad (`CopyText`, `SetSelection`) siguen disponibles para un servicio de accesibilidad, y sin «copiar» la contraseña de exportación hay que transcribirla, lo que empuja a elegirla más corta

14.3 Tareas de documentación de la autora

Pendientes que no tocan código pero sí los entregables académicos: P-08, P-17, P-25, P-41, P-45, P-52, P-73(a), P-85, P-88(a), P-141 — reflejar en el xlsx de requisitos y en los mockups del entregable A2-6 las redacciones y los cambios que la implementación introdujo por camuflaje o por decisión posterior (entre ellos el retiro de la pestaña «Lugares» de la fachada).

14.4 Riesgos residuales aceptados

No son tareas: son límites conocidos del prototipo que deben declararse en el informe. Están consolidados en `docs/PENDIENTES.md` (P-22, P-33, P-37, P-44, P-48, P-56, P-60, P-

62, P-70, P-76, P-86, P-110) y resumidos en `docs/RESUMEN_SEGURIDAD_TESIS.md`. Los de mayor alcance:

- La caché de contenido de Firestore guarda los textos de orientación en claro en el almacenamiento privado de la aplicación, y la salida rápida no la limpia: un acceso forense con root los leería.
- La contraseña de la cuenta es la raíz de secreto del respaldo. Quien obtenga los blobs de la nube puede intentar fuerza bruta offline; PBKDF2 con 600 000 iteraciones la encarece, pero no la elimina. No hay medidor de fortaleza de contraseña.
- La llave de datos no se rota al cambiar la contraseña, por diseño (para no recifrar todo): un sobre viejo exfiltrado más la contraseña vieja siguen abriendo el respaldo vigente.
- El desempate entre dos ediciones hechas en dos dispositivos depende del reloj de cada teléfono.
- La biblioteca de PDF (PdfBox-Android 2.0.27.0) no publica versiones desde enero de 2023. Aurora solo escribe con ella y con BouncyCastle excluido, pero como los PDF adjuntos se anexan, su parser corre sobre archivos que la usuaria eligió (P-75).

Glosario

ADR. Registro de una decisión de arquitectura con su motivo. Los del proyecto están en docs/ARQUITECTURA.md.

APK de depuración y de publicación. Los dos instalables de la aplicación (debug y release). El de publicación pasa por R8 y por el guardián de camuflaje.

Claim admin. Atributo personalizado de la cuenta de Firebase que identifica a la administradora. Las reglas de Firestore lo exigen para escribir contenido.

Código de acceso. Secreto que la usuaria escribe en la barra de búsqueda para entrar al espacio protegido. Se guarda solo como resumen criptográfico y no sale del teléfono.

Código de activación. Código que habilita el primer acceso y la creación de la cuenta.

Contraseña de la cuenta. Contraseña de la cuenta de Firebase. Protege el respaldo en la nube y permite recuperarlo desde otro teléfono.

DEK (llave de datos). Llave aleatoria que cifra los registros y los adjuntos antes de subirlos. En la nube viaja envuelta con una llave derivada de la contraseña de la cuenta.

Espacio protegido. Parte interna de la aplicación, con la Guía, el cuestionario y el registro.

Fachada. Parte de clima de la aplicación, visible para cualquier persona.

FLAG_SECURE. Indicador de ventana de Android que bloquea las capturas de pantalla y oculta el contenido en la vista de aplicaciones recientes.

Grafo seguro. Conjunto de pantallas del espacio protegido en Navigation Compose. Todas heredan FLAG_SECURE.

Guardián de camuflaje. Verificación de la compilación de publicación que falla si el APK contiene términos que delatarían el propósito de la aplicación.

Keystore. Almacén de llaves del sistema Android. Las llaves que guarda no se pueden exportar.

Llave de servicio. Credencial con permisos de administración del proyecto de Firebase que usan los scripts. Nunca se guarda dentro del repositorio.

MVVM. Patrón de arquitectura (modelo, vista y modelo de vista) con el que se organiza la aplicación en capas.

P-NN. Identificador de un pendiente o riesgo registrado en docs/PENDIENTES.md.

PBKDF2. Función que deriva una llave a partir de una contraseña repitiendo el cálculo muchas veces (600 000 en Aurora), para encarecer los ataques de fuerza bruta.

Pruebas instrumentadas. Pruebas que corren en un emulador o en un dispositivo, a diferencia de las unitarias, que corren en el computador.

Puerta de acceso. Componente que compara lo escrito en la barra de búsqueda con el resumen criptográfico del código de acceso.

Reglas de Firestore. Archivo firestore.rules que define quién puede leer y escribir cada colección de la base de datos.

Siembra. Carga del contenido inicial en Firestore con los scripts de tools/seed.

SQLCipher. Extensión de SQLite que cifra por completo la base de datos local.

TRL. Nivel de madurez tecnológica (Technology Readiness Level) con el que se mide el avance del prototipo.

Referencias

Google. (2026a). *Documentación de Firebase: Authentication, Cloud Firestore y reglas de seguridad*. https://firebase.google.com/docs

Google. (2026b). *Guía de arquitectura de apps*. Android Developers. https://developer.android.com/topic/architecture

International Organization for Standardization, International Electrotechnical Commission & IEEE. (2022). *ISO/IEC/IEEE 26514:2022. Systems and software engineering — Design and development of information for users*. ISO.

International Organization for Standardization & International Electrotechnical Commission. (2022). *ISO/IEC 27001:2022. Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. ISO.

Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. (2012). Congreso de la República de Colombia.

OWASP Foundation. (2024). *OWASP Mobile Application Security Verification Standard (MASVS)* (Versión 2.1.0). https://mas.owasp.org/MASVS/

Zetetic LLC. (2025). *SQLCipher for Android*. https://www.zetetic.net/sqlcipher/

Anexos digitales

Los siguientes archivos acompañan este documento. El código y su documentación están en el repositorio github.com/leydymf/aurora-app, en la etiqueta v0.3.4-trl4; el instalable está en la carpeta «anexos digitales/fase 4» de la entrega, publicada en github.com/leydymf/aurora-entregables.

Archivo	Contenido
docs/MANUAL_TECNICO.md (repositorio)	Fuente de este manual, en Markdown, junto al código que describe
docs/ARQUITECTURA.md (repositorio)	Arquitectura y registro de decisiones (ADR) con el motivo de cada una
docs/REQUERIMIENTOS.md (repositorio)	Especificación de desarrollo de los 46 requerimientos funcionales y 11 no funcionales
docs/PLAN_PRUEBAS.md y docs/RESULTADOS_PRUEBAS.md (repositorio)	Plan de pruebas y resultados de cada corrida
docs/PENDIENTES.md (repositorio)	Registro de pendientes y riesgos residuales (P-NN) que cita este manual
aurora-clima-0.3.4.apk	Instalable de depuración de la versión v0.3.4-trl4 (SHA-256 e8a28b98…1da3559)